

M11 / LEARNER BOOK

Storage, Parquet and lakehouse foundations

Standalone Zero-to-Job-Ready Data Engineer 2026

10 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE11-01 | Row vs columnar storage; where Parquet fits 4

DE11-02 | Parquet schema, row groups and metadata 7

DE11-03 | Projection, predicates and analytical reads 10

DE11-04 | Compression, small files and compaction 13

DE11-05 | Partitioning strategy and over-partitioning 16

DE11-06 | Lake vs warehouse vs lakehouse 19

DE11-07 | Delta tables, transaction log and ACID concepts 22

DE11-08 | MERGE/upsert and current-state records 25

DE11-09 | Schema enforcement, controlled evolution and compatibility 28

DE11-10 | Time travel, retention, VACUUM safety and medallion checkpoint 31

START HERE

How to use this module

Open the matching practice folder from the course root: 10_M11/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is the teacher. M11 is a foundations module: understand the storage/table mental models deeply now; M12 adds distributed Spark execution and M13 later deepens Delta/lakehouse implementation.

Runtime honesty: the pure-Python simulators and static controls pass, but native PyArrow/Delta/Spark runtime is a separate learner-PC proof. Do not treat simulator success as real engine execution.

Practice: 10_M11/practice/ Quick revision:

12_REVISION_AND_TESTS/M11/M11_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M11/PROTECTED_OPEN_AFTER_ATTEMPT/M11_Module_Revision_Test_Answers.pdf

DE11-01

Row vs columnar storage; where Parquet fits

Watch first

[1. Visual: Data Mozart - Parquet File Format Explained](#)

03:06-05:43 row vs column + row groups

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Storage format is not a cosmetic file-extension choice. It changes how much data an analytical engine must read, how types are represented, how well repeated values compress, and whether later tools can skip work. CSV remains useful for interchange and inspection, but an analytics pipeline that repeatedly scans a few columns from wide data benefits from a columnar format such as Parquet.

Picture it in your head

Picture eight order cards. A row-oriented layout keeps every field for card 1 together, then every field for card 2. A columnar layout groups all order IDs, then all dates, then all regions, then all amounts. Parquet is not literally one file per column: it splits rows into row groups and stores each column together inside each row group. That hybrid detail matters later when we discuss skipping.

Words before we use them

row-oriented = values for one record stay together; columnar = values for one field stay together within a storage unit; OLTP = workloads that often read/update individual records; OLAP = analytical scans/aggregations; Parquet = self-describing columnar file format; projection = choose only required columns.

Useful visual / current reference

Visual: Data Mozart - Parquet File Format Explained 03:06-05:43 row vs column + row groups

[OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - Read and write Parquet files Columnar format, projection, schema, partitions [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/query/formats/parquet>

Follow with me once

Open `practice/data/orders.csv`. It has six business columns and eight order rows. Imagine a report that needs only region and amount. A simple CSV reader still parses every field in every row. A Parquet-capable engine can request only the column chunks it needs. In the pure-Python simulator we count this as a conceptual 48 cell-values available versus 16 projected values. That count is a mental model, not a benchmark claim about bytes or speed.

```
python practice/storage_sim/storage_reasoning_simulator.py --section layout # Optional learner-PC real
Parquet lab after installing pyarrow: python practice/parquet/parquet_lab.py --section build
```

What you should see

The simulator reports 8 rows x 6 columns = 48 logical source cells and a two-column projection of 16 logical values. The optional PyArrow lab writes a real Parquet file from the same fixture and prints its schema. Do not claim a performance multiplier from this tiny classroom dataset.

Break it / common failure

The tempting slogan 'Parquet is always faster' is too broad. Reading a single tiny record, ingesting streaming bytes, or exchanging data with a simple external system can favor other formats. The engineering question is workload + access pattern + interoperability, not fashion.

Reason through a realistic incident

A teammate converts a 20 KB configuration export to Parquet because 'all data engineering uses Parquet.' The pipeline becomes more complex without reducing meaningful cost. Explain why format choice should follow access pattern and scale.

Deepen the engineering reasoning

A useful way to choose a storage format is to separate three questions. First, what is the unit of access: one record, a small keyed set, or a scan across many records? Second, which fields are usually needed: nearly all columns or only a narrow subset? Third, who must read the data and with which tools? A transactional database can be excellent for point updates even though a columnar file is excellent for analytical scans. CSV can be a sensible interchange format even though it lacks Parquet's typed column metadata. Parquet earns its place when analytical engines repeatedly scan large datasets and can benefit from projection, compression and metadata-aware reads. That is why 'Parquet is faster' is an incomplete sentence; faster for which operation, on which engine, at what scale? In this module, keep logical correctness separate from physical efficiency. If two files contain the same eight valid orders and reconcile to the same 1100 amount total, they can be logically equivalent while having very different scan cost. Later, Spark will make that distinction visible through distributed reads. For now, the professional habit is to write the workload before choosing the format: expected row count, columns commonly selected, update pattern, retention, interoperability and the engine that will consume it. This prevents format choice from becoming cargo-cult architecture.

Why the professional version is safer

Use Parquet when typed columnar analytical reads, compression and ecosystem interoperability matter. Keep source/raw evidence when the pipeline contract requires it; converting to Parquet is a new representation, not proof that the original source is no longer needed.

Try it yourself

For three workloads - point lookup of one customer, nightly sum of amount by region across millions of rows, and a vendor handoff to a nontechnical partner - choose row-oriented database/CSV/Parquet and defend the trade-off. Then state what Parquet buys the nightly analytical case.

Fresh changed retry

A report now needs all six columns for only two rows rather than two columns for all rows. Explain why the columnar advantage is less obvious and why file format alone cannot answer the performance question.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Use Parquet when typed columnar analytical reads, compression and ecosystem interoperability matter. Keep source/raw evidence when the pipeline contract requires it; converting to Parquet is a new representation, not proof that the original source is no longer needed.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-02

Parquet schema, row groups and metadata

Watch first

[1. Visual: Data Mozart - Parquet File Format Explained](#)

05:01-07:30 row groups + projection/predicate

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Calling Parquet 'columnar' is not enough to debug real reads. A Parquet file contains row groups; each row group contains one column chunk per column; column chunks contain pages; the footer stores schema and metadata that let readers locate data and inspect statistics. This structure is why a reader can avoid touching some bytes.

Picture it in your head

Picture a library with two floors. Each floor contains shelves for order_id, date, region and amount for only the rows assigned to that floor. The catalogue at the exit lists where each shelf is and summary facts such as the minimum and maximum amount. The reader opens the catalogue first, then visits only relevant shelves/floors.

Words before we use them

row group = horizontal batch of rows inside a Parquet file; column chunk = one column's values for one row group; page = smaller encoded/compressed unit inside a column chunk; footer = file metadata written after data; statistics = metadata such as min/max/null count when available.

Useful visual / current reference

Visual: Data Mozart - Parquet File Format Explained 05:01-07:30 row groups + projection/predicate [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Apache Parquet file format Row groups, column chunks, footer metadata [OPEN SOURCE - CLICK HERE](#)

<https://parquet.apache.org/docs/file-format/>

Follow with me once

Our metadata fixture deliberately models two row groups: RG0 covers 2026-09-01 through 2026-09-02 with amount 80-200; RG1 covers 2026-09-03 through 2026-09-04 with amount 90-190. A date filter `>= 2026-09-03` can prove RG0 is impossible. An amount filter `> 195` can prove RG1 is impossible. The fixture is clearly labelled as a course metadata simulation; the optional PyArrow lab inspects real metadata when PyArrow is available.

```
python practice/storage_sim/storage_reasoning_simulator.py --section metadata
python practice/parquet/parquet_lab.py --section metadata # requires pyarrow
```

What you should see

The simulator prints two row groups and identifies which group can be skipped for each supplied predicate. On the learner PC, the real Parquet lab should show 8 rows, 2 row groups (row_group_size=4), schema fields, and per-row-group amount/date statistics when written by the pinned PyArrow runtime.

Break it / common failure

Min/max metadata is not a magical index. If each row group contains the full value range, min/max cannot exclude anything. Statistics can also be absent or affected by writer/format details. Never promise data skipping from a file extension alone.

Reason through a realistic incident

A query filters amount `> 150`, but every row group has min 1 and max 10,000 because records arrived randomly. The team expected row-group skipping and gets little. Diagnose the difference between 'metadata exists' and 'metadata is selective.'

Deepen the engineering reasoning

Parquet metadata is useful because it lets an engine reason about a file before decoding every value. Think in layers. The file has a schema that says which fields exist and their types. The file is divided into row groups; each row group covers a horizontal slice of rows. Inside a row group, each column is stored as a column chunk. Metadata can include row counts, encodings and statistics such as minimum, maximum and null count. Those statistics are not the business truth by themselves; they are evidence an engine may use to avoid work. In our fixture, row group 0 covers dates 2026-09-01 through 2026-09-02 and row group 1 covers 2026-09-03 through 2026-09-04. A filter for dates from 2026-09-03 onward can therefore identify row group 0 as impossible before reading its values. But statistics have limits: missing statistics, broad min/max ranges, encrypted metadata, unsupported predicates or data distributions can prevent useful skipping. Also distinguish schema from business contract. A Parquet field typed as string does not prove that every customer_id is a valid customer. Physical metadata helps readers plan work; quality rules still belong to the pipeline. When you inspect metadata, record what it can prove, what it only suggests and what still requires reading/validation. That three-column habit - prove / suggest / cannot answer - is a practical way to avoid overclaiming from file metadata.

Why the professional version is safer

Treat row-group statistics as observable evidence used by a reader/planner, not as a guarantee. Inspect real file metadata when diagnosing scan behavior and keep the logical predicate separate from the physical skipping opportunity.

Try it yourself

Using `parquet_metadata_fixture.json`, decide which row groups could be skipped for (a) ``order_date` >= 2026-09-03`, (b) `amount > 195`, and (c) `region = 'East'`` when the fixture has no region min/max. Explain exactly what evidence supports each decision.

Fresh changed retry

Change the amount predicate to `amount >= 90`. Re-evaluate both row groups and explain why a less selective boundary changes skipping even though the physical file did not change.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Treat row-group statistics as observable evidence used by a reader/planner, not as a guarantee. Inspect real file metadata when diagnosing scan behavior and keep the logical predicate separate from the physical skipping opportunity.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-03

Projection, predicates and analytical reads

Watch first

[1. Visual: Data Mozart - Parquet File Format Explained](#)

05:43-07:30 projection + predicates

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Analytical efficiency usually combines two separate ideas: projection reduces columns read; predicate/data skipping reduces chunks or row groups considered. Beginners often mix these up and say a WHERE clause means the engine read only matching rows. It might still read a row group and filter rows after decode.

Picture it in your head

Projection is choosing which shelves to visit. Predicate skipping is deciding an entire floor cannot possibly contain the requested book. Row-level filtering is checking individual books after entering a floor. Those are three different stages.

Words before we use them

column pruning/projection = do not read unrequested columns; predicate pushdown = pass a filter toward the data source/reader; data skipping = avoid files/row groups using metadata; residual filter = condition still evaluated after data is read/decoded.

Useful visual / current reference

Visual: Data Mozart - Parquet File Format Explained 05:43-07:30 projection + predicates [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - Read and write Parquet files Column projection and row-group skipping context [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/query/formats/parquet>

Follow with me once

For `SELECT order_id, amount FROM orders WHERE order_date >= '2026-09-03'`, the desired columns are `order_id`, `amount`, plus whatever the engine needs to evaluate `order_date`. The fixture says RG0 ends on Sep 2, so it can be skipped; RG1 remains. This does not mean only final qualifying rows were physically read from RG1.

```
python practice/storage_sim/storage_reasoning_simulator.py --section reads # Learner-PC PyArrow projection
```

```
example: python practice/parquet/parquet_lab.py --section projection
```

What you should see

The simulator separates `columns_requested` from `row_groups_candidate`. For the date predicate, RG0 is excluded and RG1 remains. The optional PyArrow read returns only the requested output columns while the filter behavior is inspected separately.

Break it / common failure

A common wrong proof is timing one tiny local query and concluding predicate pushdown works. OS cache, tiny data, engine overhead and different physical layout can dominate. Better evidence is metadata, query plan/scan metrics on a capable engine, and controlled comparisons.

Reason through a realistic incident

A dashboard selects only `order_id` and `amount`, but a developer wraps the Parquet read in code that first loads every column into pandas and then slices two columns. Explain why the final DataFrame shape does not prove column pruning occurred.

Deepen the engineering reasoning

Projection and predicate pushdown are often mentioned together, but they solve different kinds of wasted work. Projection reduces the columns requested. Predicate pushdown lets a filter reach the scan

layer so the reader can use partition or file statistics to avoid data that cannot match. Row-group skipping is one possible consequence when metadata proves a whole row group fails the predicate. Imagine a report that needs `order_id` and `amount` for dates on or after 2026-09-03. Projection asks for two output columns, although the filter column `order_date` may still be read or inspected to evaluate the filter. Predicate reasoning compares the date condition with metadata and may skip earlier row groups. The important boundary is that 'pushdown happened' and 'data was skipped' are not synonyms. An engine can push a predicate but still need to read every group because the metadata ranges overlap the filter. Conversely, partition pruning can skip whole directories before Parquet row-group logic starts. When debugging a slow read, ask in order: Did I select unnecessary columns? Is the filter expressed in a form the engine can push? Does the physical layout provide useful statistics or partitions? Is the dataset so small that overhead dominates anyway? In later Spark work you will inspect plans and metrics. Here, mastery means you can predict which physical units are candidates and then validate that prediction instead of treating optimization terms as magic incantations.

Why the professional version is safer

Design schemas and file layout so common analytical filters have useful metadata and common projections avoid wide reads. Verify with engine evidence; SQL syntax expresses intent but does not by itself prove physical I/O.

Try it yourself

For three queries, list output columns, filter columns, candidate row groups from the fixture, and one piece of evidence that would prove the engine actually pruned work. Do not use runtime alone as the only proof.

Fresh changed retry

Remove the date predicate but keep only two projected columns. Explain what optimization remains and what optimization disappears.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Design schemas and file layout so common analytical filters have useful metadata and common projections avoid wide reads. Verify with engine evidence; SQL syntax expresses intent but does not by itself prove physical I/O.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-04

Compression, small files and compaction

Watch first

[1. Visual: Data Mozart - Parquet File Format Explained](#)

07:30-10:04 file size + compression

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Compression and compaction solve different problems. Compression reduces bytes inside files. Compaction reduces the number of files by rewriting many small files into fewer larger files. A directory containing 10,000 tiny, well-compressed files can still be expensive to list, plan and open.

Picture it in your head

Compression is vacuum-packing each suitcase. Compaction is replacing hundreds of tiny suitcases with a few appropriately sized ones. Vacuum-packing does not reduce how many suitcases the airport must tag and handle.

Words before we use them

codec = compression algorithm such as Snappy/ZSTD/GZIP; encoding = representation such as dictionary/RLE before/alongside compression; small-file problem = operational/query overhead from many tiny files; compaction = rewrite into fewer larger files; OPTIMIZE = Databricks table-layout rewrite operation.

Useful visual / current reference

Visual: Data Mozart - Parquet File Format Explained 07:30-10:04 file size + compression [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - Optimize data file layout Compaction/file-layout maintenance [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/operations/optimize>

Follow with me once

The supplied manifest contains six tiny files totaling 65 MB. The compaction planner groups them into one logical target under a 128 MB classroom target. The output explicitly says it is a planning simulation; it does not pretend OPTIMIZE ran. Current Databricks docs also note that an OPTIMIZE rewrite creates new files while older unreferenced files can remain physically present until retention/VACUUM permits cleanup.

```
python practice/storage_sim/storage_reasoning_simulator.py --section compaction # Read
practice/data/small_files_manifest.json
```

What you should see

The planner reports 6 source files / 65 MB and one proposed compacted group. Logical row/amount controls must remain identical before and after the plan. The plan changes physical layout, not business content.

Break it / common failure

Do not set one universal 'perfect file size' and cargo-cult it across engines/workloads. Target sizes, automatic tuning and clustering features vary. Also do not call deleting files 'compaction' - compaction is a rewrite preserving logical data.

Reason through a realistic incident

An engineer gzips 5,000 tiny JSON files and celebrates a 70% storage reduction, yet Spark query planning remains slow. Explain why fewer bytes and fewer file-open/listing operations are separate axes.

Deepen the engineering reasoning

Small-file problems are a systems problem, not merely a file-size aesthetic. Each file can add listing, planning, metadata and open/close overhead. Thousands of tiny files can therefore make a dataset expensive to discover and schedule even when the total number of bytes is modest. Compaction rewrites many small files into fewer larger files while preserving the logical table content. That preservation requirement is why reconciliation belongs in a storage lesson: row count, key count and trusted amount total must match before and after the rewrite. Compression is related but different. Compression reduces bytes using encodings/codecs; compaction changes file layout and file count. A compressed tiny-file swarm can still be a small-file problem. Likewise, one gigantic file can reduce file count but hurt parallelism, recovery granularity or write behavior. The correct target is workload- and platform-dependent. Our classroom manifest deliberately uses six tiny logical files totaling 65 MB and a 128 MB teaching target to make the reasoning visible, not to decree a universal production number. In a real platform, inspect the engine's recommendations, workload concurrency, partitioning, write frequency and downstream scan sizes. Also remember that table formats may keep old files after compaction for time travel until retention cleanup. Seeing storage usage temporarily increase after OPTIMIZE-like maintenance is therefore not automatically a leak. First identify active files versus retained historical files, then reconcile the logical table.

Why the professional version is safer

Measure file count, typical file size, partition distribution and query behavior before tuning. Prefer platform-managed optimization where appropriate, but understand the invariant: physical

rewrites must preserve logical rows/keys/totals.

Try it yourself

Given 40 files of 3 MB and 4 files of 600 MB, propose separate actions for the two groups and name one risk of compacting everything into a single huge file.

Fresh changed retry

Assume the table is partitioned by day and the small files are spread across 30 daily partitions. Explain why compaction may be constrained within partition boundaries and why over-partitioning can preserve the small-file problem.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Measure file count, typical file size, partition distribution and query behavior before tuning. Prefer platform-managed optimization where appropriate, but understand the invariant: physical rewrites must preserve logical rows/keys/totals.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-05

Partitioning strategy and over-partitioning

Watch first

[1. Optional visual: Data Mozart - Parquet File Format Explained](#)

05:01-10:04 row groups/projection/file-size context; this video does NOT teach partition strategy. Official partition docs + book are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Partitioning can eliminate entire directory/table slices when filters align with the partition key, but too many partitions create metadata and small-file overhead and can prevent compaction across boundaries. A timestamp/customer/order ID is usually a poor partition key because its cardinality can explode.

Picture it in your head

A partition is a labeled room. 'year=2026/month=09' creates a few useful rooms. 'customer_id=C000001' creates a room for every customer. If most rooms contain one shoebox, the warehouse is organized but inefficient.

Words before we use them

partition column = field used to divide physical storage; cardinality = number of distinct values; partition pruning = skip partitions using partition values; over-partitioning = too many tiny partitions; clustering = alternative physical organization used by some platforms.

Useful visual / current reference

Optional visual: Data Mozart - Parquet File Format Explained 05:01-10:04 row groups/projection/file-size context; this video does NOT teach partition strategy. Official partition docs + book are required. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - When to partition tables Partitioning vs over-partitioning; current alternatives [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/partitions>

Follow with me once

Our eight-row fixture has four distinct dates and four regions but eight order IDs. The simulator compares partition counts for date, region and order_id and calculates rows per partition. The point is not that date is always correct - only that high-cardinality keys can create one-row directories quickly. Current Databricks guidance increasingly prefers liquid clustering for many managed tables, so partitioning should be a deliberate choice rather than an automatic ritual.

```
python practice/storage_sim/storage_reasoning_simulator.py --section partitioning
```

What you should see

The simulator shows 4 date partitions, 4 region partitions and 8 order_id partitions for only 8 rows. order_id therefore has one row/partition in the fixture. You should be able to explain why that pattern scales badly.

Break it / common failure

Do not choose a partition key only because users filter it sometimes. Estimate distinct values, data per partition, filter selectivity, write pattern and maintenance behavior. A high-cardinality timestamp can create vast numbers of tiny directories/files.

Reason through a realistic incident

A table is partitioned by event_timestamp down to seconds. Every batch creates thousands of almost-empty partitions, and compaction cannot combine across them. Explain the root design error and a safer alternative category.

Deepen the engineering reasoning

Partitioning is valuable when a filter can eliminate large groups of data cheaply, but a partition key is not automatically good because it appears in WHERE clauses. Cardinality and distribution matter. Partitioning eight classroom rows by order_id would create eight nearly empty partitions: the directory layout becomes an index made of folders, with overhead and no useful grouping. A date key can also be too fine if the table is small or the workload rarely filters by date. Region has lower cardinality in our fixture, but that still does not make it universally best; if nearly every query scans all regions, the partition directories add complexity without pruning benefit. A professional partition decision includes expected table size, common filters, cardinality, skew, ingestion pattern and maintenance behavior. Modern lakehouse engines can also use clustering or data-skipping features that reduce the need for rigid directory partitioning, so avoid copying old Hive-era rules blindly. Before creating a partition column, estimate how many partitions exist per retention window and approximately how much data lands in each. Then imagine the smallest common query: how many partitions would it touch? Finally consider writes: will late data or updates repeatedly hit many partitions? If the answers look messy, test an unpartitioned or differently clustered design. Partitioning is physical layout, not a business grain. Changing it should not alter the row/key/amount truth of the table; always keep those logical controls separate.

Why the professional version is safer

Partitioning is a physical optimization contract. Keep partition cardinality and files-per-partition observable; do not let an old layout silently become a new bottleneck as volume and query patterns change.

Try it yourself

Choose among no partitioning, date, region, customer_id and order_id for a hypothetical 2 TB order table. State what workload facts you would demand before deciding. Your answer must include a cardinality/file-size check, not just a query-filter example.

Fresh changed retry

The table grows from 50 GB to 5 TB and common filters move from region to date ranges. Explain why a physical-layout decision should be revisited instead of treated as permanent truth.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Partitioning is a physical optimization contract. Keep partition cardinality and files-per-partition observable; do not let an old layout silently become a new bottleneck as volume and query patterns change.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-06

Lake vs warehouse vs lakehouse

Watch first

[1. Optional visual: Data Mozart - Parquet File Format Explained](#)

10:04-11:28 Delta-format introduction only; lakehouse architecture is taught by the book + current official docs.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

A data lake, warehouse and lakehouse are architecture patterns with different storage/management assumptions, not job titles or product slogans. The useful question is where raw/open files live, where governed tables live, what transaction/quality guarantees exist, and which engines can read the data.

Picture it in your head

A lake is a large storage yard that can keep many shapes of raw cargo. A warehouse has catalogued shelves and controlled receiving rules. A lakehouse adds a table-management layer over object/file storage so data can retain open-file economics while gaining stronger table semantics. Real platforms blend these ideas, so the boundaries are architectural, not perfectly pure.

Words before we use them

data lake = scalable file/object storage for varied data; warehouse = managed analytical database/table system; lakehouse = architecture adding warehouse-like table management/transactions/governance over lake-style storage; open format = format readable by multiple compatible engines; medallion = Bronze/Silver/Gold layering pattern.

Useful visual / current reference

Optional visual: Data Mozart - Parquet File Format Explained 10:04-11:28 Delta-format introduction only; lakehouse architecture is taught by the book + current official docs. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=5NA57Pfpdr4>

Official: Databricks - What is a data lakehouse? Lake vs warehouse vs lakehouse role [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/lakehouse/>

Follow with me once

Use one pipeline: vendor CSV lands unchanged, standardized typed records become Silver-like data, and a business aggregate becomes Gold-like serving data. In a lake-only design those may be files plus conventions. In a warehouse, managed tables and SQL engine rules dominate. In a lakehouse, Parquet-backed table formats such as Delta add a transaction/table layer over object storage.

```
python practice/storage_sim/storage_reasoning_simulator.py --section architecture
```

What you should see

The simulator prints the same data flow under three architecture lenses and asks which layer owns schema, transactions and serving contracts. There is no score for choosing a fashionable label; the answer must state storage, table semantics and consumers.

Break it / common failure

Do not claim 'lakehouse replaces every warehouse' or 'lake means ungoverned mess.'
Governance and modeling depend on implementation. Also do not call a folder of Parquet files a transactional table merely because a SQL engine can query them.

Reason through a realistic incident

A team says its S3 folder of Parquet files is a lakehouse because BI can query it. No transaction log or managed table state exists. Explain what capability is missing from that claim and what could still be true about the folder.

Deepen the engineering reasoning

Lake, warehouse and lakehouse are architectural roles, not three mutually exclusive product boxes. A data lake traditionally emphasizes low-cost object/file storage and open or semi-open file formats, making it easy to retain raw and diverse data. A warehouse traditionally emphasizes managed tables, SQL semantics, governance and strong performance for curated analytics. A lakehouse tries to keep

object/file storage economics and openness while adding table semantics such as transactions, schema controls, history and governance. The mistake is to reduce the comparison to 'cheap versus fast' or 'old versus new.' Real organizations often use more than one pattern, and cloud warehouses themselves can read external files while lakehouse systems can expose warehouse-like SQL endpoints. For a design decision, ask what guarantees are required: concurrent writers, transaction consistency, data sharing, fine-grained governance, BI latency, open-format access, multi-engine interoperability and operational skill. Then separate storage from the table/catalog layer above it. In this course, Parquet explains physical files; Delta-like table semantics explain how a set of files can behave more like a reliable table. That distinction prepares you for Spark and Delta without pretending every Parquet directory is a lakehouse table. A folder of Parquet files can be valuable analytical storage and still lack atomic multi-file commits, version history and safe MERGE semantics unless an appropriate table layer supplies them.

Why the professional version is safer

Choose architecture by guarantees, workloads, interoperability, cost and operating skill. Keep raw evidence, curated contracts and serving models distinct even when one platform exposes them through a unified UI.

Try it yourself

For a small company with raw event archives, curated finance tables and a BI workload, sketch lake, warehouse and lakehouse options. For each, state where raw bytes live, how table correctness is enforced and how analysts query serving data.

Fresh changed retry

Assume regulations require immutable raw evidence and strong audited serving tables. Explain how a hybrid architecture might intentionally keep both raw lake storage and governed warehouse/lakehouse tables.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Choose architecture by guarantees, workloads, interoperability, cost and operating skill. Keep raw evidence, curated contracts and serving models distinct even when one platform exposes them through a unified UI.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-07

Delta tables, transaction log and ACID concepts

Watch first

[1. Optional visual: Databricks - Diving into Delta Lake: Transaction Log](#)

No independently verified bounded chapter range assigned; book + current official Delta docs are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Parquet gives efficient files but not, by itself, a safe multi-file table transaction. Delta Lake adds a transaction log that records which data files constitute each committed table version. Readers use a consistent snapshot; writers commit changes atomically through the log rather than asking every reader to infer table state from a directory listing.

Picture it in your head

Imagine a restaurant inventory made of boxes plus an official ledger. The boxes are Parquet data files. The ledger says which boxes are active in version 0, then which boxes were added/removed in version 1. A reader trusts one committed ledger page, not whatever loose boxes happen to be visible mid-delivery.

Words before we use them

transaction log = ordered record of table actions/commits; snapshot = table state at one committed version; ACID = atomicity, consistency, isolation, durability; optimistic concurrency = writers validate/commit against shared state; data file = physical Parquet file referenced by table state.

Useful visual / current reference

Optional visual: Databricks - Diving into Delta Lake: Transaction Log No independently verified bounded chapter range assigned; book + current official Delta docs are required. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=IXIBjdehsdw>

Official: Databricks - What is Delta Lake? Transaction log, ACID, schema enforcement, metadata [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/delta>

Follow with me once

Run the Delta-like simulator. Version 0 points to `part-v0.json` representing eight orders totaling 1100. A merge creates version 1 and writes a new snapshot totaling 1250. The simulator writes JSON log records so you can see `add/remove/version` actions. It is deliberately named a simulator: it teaches the log/snapshot model but is not the Delta Lake engine.

```
python practice/storage_sim/delta_table_simulator.py init python
practice/storage_sim/delta_table_simulator.py merge python practice/storage_sim/delta_table_simulator.py
history
```

What you should see

History shows version 0 INIT and version 1 MERGE. Current state has 9 unique order IDs and total amount 1250. Version 0 remains queryable inside the simulator until cleanup rules remove the old snapshot fixture.

Break it / common failure

Never edit Parquet files or `_delta_log` internals by hand as a normal repair strategy. A table format's guarantees depend on coordinated log + data-file semantics. Random file deletion can corrupt the table or destroy time-travel capability.

Reason through a realistic incident

An operator deletes an 'old-looking' Parquet file directly from a Delta table directory to save storage. A historical query later fails. Explain why the filesystem view was not sufficient evidence that the file was safe to delete.

Deepen the engineering reasoning

The transaction log is the key mental bridge from 'files in a directory' to 'a table with versions.' In a Delta-style design, a successful write does not mean 'overwrite some Parquet files and hope readers catch a consistent moment.' Instead, the table records an ordered commit describing which data files are active, which were removed from the current snapshot, and metadata about the operation. Readers reconstruct a table version from committed log state. That is why a new snapshot can become visible atomically even though multiple physical files may be involved. ACID still needs precise language. Atomicity means a commit is visible as a unit; consistency means declared invariants/constraints and table rules are preserved; isolation concerns concurrent operations seeing sensible states; durability means committed state survives expected failures. Do not claim that a transaction log makes every upstream business rule correct. The log can faithfully commit duplicate customer IDs if your pipeline allows them. Our simulator intentionally records version, operation, active snapshot, row count, unique keys and amount total so you can see the control plane, but it is not the Delta engine. Use it to reason about commit boundaries and history. Later, when running real Delta, compare the engine's history/log behavior with this mental model and note where implementation details are richer than the simulator.

Why the professional version is safer

Treat the transaction log as table metadata/state, not optional bookkeeping. Use supported table APIs/commands so file rewrites, metadata and versions stay coherent.

Try it yourself

Draw version 0 and version 1 as sets of active files plus log commits. Explain atomicity in one sentence and state what a reader should observe if a writer fails before committing version 2.

Fresh changed retry

Two writers start from version 1 and both want to update the same business keys. Explain why a transaction protocol needs conflict detection/validation instead of letting both silently overwrite directory contents.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Treat the transaction log as table metadata/state, not optional bookkeeping. Use supported table APIs/commands so file rewrites, metadata and versions stay coherent.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-08

MERGE/upsert and current-state records

Watch first

[1. Optional visual: Databricks - Deep-Dive into Delta Lake](#)

No independently verified bounded chapter range assigned; MERGE/source-key safety comes from the book + current official docs.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

MERGE is a table-level way to reconcile a source change set with current target state: matched keys can update/delete and unmatched keys can insert. The hardest part is not syntax; it is proving the source has an unambiguous row for each target key and that rerunning the change set produces the intended state.

Picture it in your head

Picture a clerk with a current customer/order card file and a change envelope. For each change, the clerk finds the matching card or creates a new one. If the envelope contains two contradictory changes for the same card, the clerk needs a rule before touching the cabinet.

Words before we use them

upsert = update existing key or insert new key; MERGE = conditional matched/not-matched table operation; business key = identity used for match; source deduplication = ensure source row/key rule is unambiguous; current-state table = one intended latest row per key.

Useful visual / current reference

Optional visual: Databricks - Deep-Dive into Delta Lake No independently verified bounded chapter range assigned; MERGE/source-key safety comes from the book + current official docs.

[OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=de-6a6Bfw6E>

Official: Databricks - Upsert with MERGE Matched/not-matched update/insert patterns [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/delta/merge>

Follow with me once

The M11 merge fixture updates O1003 from 200 to 215, O1005 from 90 to 95 and inserts O1009 for 130. The simulator validates source-key uniqueness before applying changes. Starting total 1100 becomes 1250 with 9 keys. A separate duplicate-source fixture contains two O1003 rows; the simulator blocks it rather than 'choosing one.'

```
python practice/storage_sim/delta_table_simulator.py reset python
practice/storage_sim/delta_table_simulator.py merge python practice/storage_sim/delta_table_simulator.py
merge-duplicate-source
```

What you should see

Clean MERGE simulation: 2 updates + 1 insert, 9 unique keys, total 1250. Duplicate-source scenario: rejected before target mutation, current version/total unchanged.

Break it / common failure

MERGE is not a license to skip source-grain checks. A source with duplicate business keys can be ambiguous and may error or produce wrong current-state logic depending on engine/rules. DISTINCT is not a universal repair because two different updates can remain genuinely conflicting.

Reason through a realistic incident

A CDC batch contains two records for O1003 with different event times. A developer drops duplicates without ordering and MERGEs the survivor. Explain why 'one row per key' is necessary but not sufficient; the survivor-selection rule must also reflect business/event semantics.

Deepen the engineering reasoning

MERGE is powerful because one statement can express current-state changes, but its safety begins before the statement runs. First define the target key and confirm the source batch has at most one

intended row per key. If source O1003 appears twice with conflicting amounts, 'last row wins' is not a neutral technical choice; it silently invents a precedence rule. That is why the simulator blocks the duplicate-key source instead of resolving it arbitrarily. Next classify matched and not-matched behavior. In our clean batch, O1003 and O1005 are updates and O1009 is an insert. The expected current state is nine unique keys totaling 1250. These three controls - update/insert counts, key uniqueness, and reconciled total - give independent evidence that the MERGE did what the business rule intended. Idempotency also needs thought. Reapplying the same current-state source should converge to the same result when the operation is designed correctly, but event-style data may require a different append/deduplicate rule. Do not use MERGE as a universal ingestion hammer. It is appropriate when the target model and source semantics support update/insert matching. In production, also reason about deletion/tombstone rules, late-arriving records, sequence/version columns and concurrent writers. A syntactically valid MERGE can be semantically wrong if the match key or precedence rule is wrong.

Why the professional version is safer

Make MERGE deterministic at the source boundary and idempotent for reruns. Reconcile affected keys/counts/totals and keep change-event evidence when the business requires history.

Try it yourself

Before writing MERGE SQL, specify target grain, match key, source deduplication/order rule, matched action, not-matched action, and three post-merge controls. Then compare target row count, unique keys and total amount to the expected fixture controls.

Fresh changed retry

Add a delete event to the source. Explain why current-state deletion policy and historical/audit requirements must be decided before adding a WHEN MATCHED . . . DELETE branch.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Make MERGE deterministic at the source boundary and idempotent for reruns. Reconcile affected keys/counts/totals and keep change-event evidence when the business requires history.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-09

Schema enforcement, controlled evolution and compatibility

Watch first

[1. Optional visual: Databricks - Deep-Dive into Delta Lake](#)

No independently verified bounded chapter range assigned; schema/evolution behavior comes from the book + current official docs.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Schema enforcement protects a table from accidental incompatible writes; schema evolution is the deliberate process of changing that contract. The safe mindset is 'fail or route unknown change until policy approves it,' not 'turn on auto-merge and accept whatever arrives.'

Picture it in your head

Think of an airport baggage gauge. Enforcement checks whether a bag meets today's allowed shape. Evolution is management changing the gauge and policy for tomorrow. Quietly sawing the gauge wider every time a bag fails is not governance.

Words before we use them

schema enforcement = reject writes that violate table contract; evolution = intentional contract change; additive column = new field; rename/drop = semantic changes that can break consumers; type widening = move to compatible broader type under supported rules; compatibility = whether existing readers/writers continue to work.

Useful visual / current reference

Optional visual: Databricks - Deep-Dive into Delta Lake No independently verified bounded chapter range assigned; schema/evolution behavior comes from the book + current official docs.

[OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=de-6a6Bfw6E>

Official: Databricks - Schema evolution Supported change classes and explicit evolution controls

[OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/data-engineering/schema-evolution>

Follow with me once

Version 1 of our current-state fixture has seven required fields. A changed source adds `source_system`; the controlled-evolution command explicitly approves that additive field and creates version 2 without changing the 1250 total. A bad fixture sends `amount='two hundred'`; the schema validator rejects it and does not advance the table version.

```
python practice/storage_sim/delta_table_simulator.py evolve-add-column python
practice/storage_sim/delta_table_simulator.py incompatible-write python
practice/storage_sim/delta_table_simulator.py history
```

What you should see

Approved additive evolution creates version 2 with `source_system` present and 9 keys / total 1250. Incompatible numeric-type input is rejected; history does not gain a successful commit for that bad write.

Break it / common failure

A new nullable column can be low-risk; renaming a field used by downstream jobs is not. Some platforms support automatic evolution for specific change classes, but support does not mean the business meaning is safe. Schema compatibility is both technical and contractual.

Reason through a realistic incident

Upstream renames `customer_id` to `client_id`. Auto-evolution treats it as a new column while the old field becomes null for new records. The pipeline keeps running but joins collapse. Explain why operational success can hide semantic breakage.

Deepen the engineering reasoning

Schema enforcement and evolution should be treated as contract management. Enforcement protects the current trusted shape; evolution changes that trusted shape deliberately. The safest workflow is detect -> classify -> approve -> migrate/test consumers -> commit the change. An additive nullable column such as `source_system` is often easier because old readers can ignore it and old rows can remain null/defaulted. A rename from `customer_id` to `client_id` is different: a naive auto-evolution can create a new field while downstream joins still reference the old name, producing valid files but invalid business results. Type changes also need semantic review. Integer to wider numeric may be technically supported, while string to struct changes how every consumer reads the value. The engine's capability is only one input to the decision. Keep a schema contract that identifies required fields, types, nullability, key semantics and owners. When a producer drifts unexpectedly, quarantine or fail the affected records/run according to policy rather than coercing values until the job turns green. Then test important downstream models. Our simulator demonstrates one approved additive field and one incompatible amount value. The important evidence is not merely that the good case succeeded and bad case failed; it is that the rejected write did not advance table version or corrupt the trusted current state. State preservation after failure is part of the contract.

Why the professional version is safer

Version schema contracts. Prefer explicit, reviewed evolution for important tables. Test downstream compatibility and keep rescue/quarantine paths for unexpected producer changes rather than silently coercing them into trusted data.

Try it yourself

Classify five changes - add nullable source_system, rename customer_id, int to double, string to struct, drop region - into 'likely additive/controlled', 'needs consumer migration', or 'incompatible/full rewrite review'. Defend each with downstream impact, not just engine capability.

Fresh changed retry

Assume a BI report selects columns by name and a streaming reader keeps a tracked schema. Explain why rename/drop planning must include consumers and not only the writer that evolves the table.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Version schema contracts. Prefer explicit, reviewed evolution for important tables. Test downstream compatibility and keep rescue/quarantine paths for unexpected producer changes rather than silently coercing them into trusted data.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime PASS.

DE11-10

Time travel, retention, VACUUM safety and medallion checkpoint

Watch first

[1. Optional visual: Databricks - Diving into Delta Lake: Transaction Log](#)

No independently verified bounded chapter range assigned; time-travel/history/VACUUM behavior comes from current official docs + book.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Time travel is possible because table versions reference historical files. Retention and VACUUM decide how long old physical files remain available. Therefore 'history row exists' and 'old version is still queryable' are related but not identical promises. Maintenance must balance audit/recovery needs against storage cleanup.

Picture it in your head

The transaction log is a photo album index; old data files are the actual photos. Keeping the index entry but shredding the photo means you can know an old version existed without being able to reconstruct every byte of it. VACUUM is a controlled cleanup, not a casual delete command.

Words before we use them

time travel = query an earlier table version/timestamp when required files remain; retention = policy for keeping old files/history; VACUUM = remove no-longer-referenced old files beyond retention; DRY RUN = preview deletion candidates; Bronze/Silver/Gold = raw-ish -> validated -> serving layer pattern.

Useful visual / current reference

Optional visual: Databricks - Diving into Delta Lake: Transaction Log No independently verified bounded chapter range assigned; time-travel/history/VACUUM behavior comes from current official docs + book. [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=IXIBjdehsdw>

Official: Databricks - Table history Versions and history [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/history>

Official: Databricks - VACUUM Retention, DRY RUN and safety [OPEN SOURCE - CLICK HERE](#)

<https://docs.databricks.com/aws/en/tables/operations/vacuum>

Follow with me once

Use simulator time travel to read version 0 total 1100 and current version 2 total 1250. vacuum-dry-run lists stale snapshot candidates but deletes nothing. The learner book then maps those ideas to current Databricks guidance: default Delta VACUUM data-file retention is 7 days, Databricks strongly recommends at least 7 days, and DRY RUN previews files. Do not reduce retention just to make a demo look dramatic.

```
python practice/storage_sim/delta_table_simulator.py time-travel 0 python
practice/storage_sim/delta_table_simulator.py current python practice/storage_sim/delta_table_simulator.py
vacuum-dry-run
```

What you should see

Version 0: 8 keys, total 1100. Current: 9 keys, total 1250. Dry run lists simulated stale files and explicitly reports deleted=false. No fixture is physically removed by the classroom command.

Break it / common failure

Do not treat VACUUM as a performance command that can be run with arbitrarily tiny retention. Removing old files can permanently eliminate time-travel/recovery capability and can be dangerous for long-running jobs. Also distinguish OPTIMIZE (rewrite layout) from VACUUM (remove eligible unused files).

Reason through a realistic incident

A team compacts a table, sees storage grow because new compacted files coexist with old ones, then immediately runs aggressive VACUUM with safety disabled. Explain the lifecycle and why the second reaction can destroy legitimate rollback/recovery windows.

Deepen the engineering reasoning

Time travel, retention and VACUUM form one lifecycle. A table version can be described by history metadata, but reconstructing that version requires the underlying data files to remain available. Retention policy therefore converts a business recovery requirement into storage behavior. If finance needs reproducible month-close states for 30 days, a seven-day file-retention window is insufficient even if seven days is a common/default platform safety threshold. The correct retention is the maximum of operational safety and business/compliance recovery needs, within platform constraints. Before cleanup, inspect history and identify long-running readers or restore requirements. Use DRY RUN where available to see candidates. After cleanup, validate the current table and document what historical recovery window remains. Also separate maintenance commands. OPTIMIZE/compaction rewrites active layout for read efficiency; VACUUM removes old no-longer-referenced files after retention. Running compaction can temporarily increase storage because both new active files and retained old files coexist. Aggressive VACUUM immediately afterward may save bytes while destroying rollback evidence. The Bronze/Silver/Gold checkpoint at the end of M11 ties these ideas together: Bronze preserves source evidence, Silver applies validated contracts and deduplication, Gold serves business-ready models. Those names are useful only when each layer has explicit grain, schema, quality, retention and reconciliation rules.

Why the professional version is safer

Make retention a business/recovery policy, not a random storage tweak. Use table-supported maintenance commands, preserve evidence of changes, and connect Bronze/Silver/Gold layers with explicit contracts and reconciliation rather than names alone.

Try it yourself

Write a maintenance runbook that includes history inspection, time-travel requirement, retention owner, VACUUM DRY RUN, safety check, and post-maintenance validation. Then sketch Bronze/Silver/Gold contracts for the M11 orders: what is preserved, what is validated, and what is served.

Fresh changed retry

Finance requires 30 days of reproducible month-close recovery instead of 7. Explain which retention decision changes and why the storage-cost trade-off is intentional rather than an error.

Validation and recovery ladder

- State the logical grain/table version/physical unit you are reasoning about before changing anything.
- Separate logical correctness from physical layout: rows/keys/totals first, then files/row groups/partitions/versions.
- Inspect the smallest evidence that can falsify your assumption: metadata fixture, file manifest, log version, source-key uniqueness or schema contract.
- Open the Explained Review only after saving a genuine attempt; reveal only the weak concept.
- Close review and solve the changed retry. Preserve the original attempt and the independent validation.

Ready-made revision

Make retention a business/recovery policy, not a random storage tweak. Use table-supported maintenance commands, preserve evidence of changes, and connect Bronze/Silver/Gold layers with explicit contracts and reconciliation rather than names alone.

Evidence / mastery

Save your command/design, actual observed output or error, one independent control, a 60-90 second explain-back, and the changed retry. If this lesson needs native Parquet/Delta-engine evidence you have not run, label that part PENDING rather than claiming a simulator/static PASS is a native runtime

PASS.